

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное
образовательное учреждение высшего образования «Тульский
государственный университет»
Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №4
по дисциплине
Интеграция и тестирование программных систем
по теме
«Тестирование API (часть 2 — автоматизация)»

Выполнил: ст. гр. 220032-11

Лаукерт К.М.

Принял:

Басалов Ю.А.

Тула 2026

Лабораторная работа №4

по теме «Тестирование API (часть 2 — автоматизация)»

Цель работы

Получить практические навыки автоматизации тестирования REST API: проектирование структуры тестового проекта, построение клиент-обёртки над HTTP-библиотекой requests (управление сессией, заголовками, токеном аутентификации, логирование), формализация контрактов ответов в виде pydantic-схем, организация фикстур PyTest (в том числе тестовых данных с очисткой), написание позитивных, негативных, граничных и параметризованных тестов, включая проверку смены состояний (CRUD), и настройка отчётности (pytest-html, JUnit XML).

Описание тестируемого API и задания

Объект тестирования — локальный REST-сервер, разработанный в ЛР2 (Flask, <http://localhost:5000>): «Сервис управления задачами» с аутентификацией по токenu. Методичка допускает использование сервера из ЛР №2. Перед прогоном тестов сервер запускается командой `py lab2_api_server.py`; готовность проверяется запросом `GET /api/v1/health`, сброс данных к исходному состоянию — `POST /api/v1/reset`.

Эндпоинт	Метод	Назначение
<code>/api/v1/auth/register</code>	POST	регистрация пользователя (email, password, name)
<code>/api/v1/auth/login</code>	POST	аутентификация, возвращает Bearer-токен
<code>/api/v1/auth/logout</code>	POST	выход (инвалидация токена)
<code>/api/v1/users/me</code>	GET	данные текущего пользователя (требуется токен)
<code>/api/v1/tasks</code>	GET, POST	список задач (фильтры completed, userId) и создание
<code>/api/v1/tasks/{id}</code>	GET, PUT, DELETE	получение, обновление, удаление задачи
<code>/api/v1/products</code>	GET	список товаров (фильтры category, minPrice, maxPrice, inStock)
<code>/api/v1/products/{id}</code>	GET	получение товара по ID
<code>/api/v1/health, /api/v1/reset</code>	GET, POST	проверка готовности, сброс данных

Задание: создать проект автоматизации на PyTest со структурой `tests/` (conftest и модули тестов по доменам), `api/` (клиент и схемы

ответов), utils/ (логирование, вспомогательные функции); реализовать клиент-обёртку requests с аутентификацией и логированием; pydantic-схемы; фикстуры (клиент, тестовые данные с очисткой, токен); тесты позитивных, негативных, граничных сценариев, смены состояний (CRUD) и параметризованные; сформировать отчёт pytest-html.

Структура тестового проекта

Проект размещён в ЛР4\api_tests (базой послужил стартовый каркас из материалов, доработанный под фактическое поведение сервера).
Назначение файлов:

Файл / каталог	Назначение
api/client.py	клиент-обёртка requests: APIClient (сессия, таймауты, Bearer-токен, логирование) и доменные классы TasksAPI, AuthAPI, ProductsAPI
api/schemas.py	pydantic-схемы ответов: Task, TaskList, DeleteTaskResponse, User, AuthResponse, RegisterResponse, ErrorResponse, Product, ProductList
tests/conftest.py	фикстуры: api_client (с ожиданием готовности сервера), autouse-сброс данных перед каждым тестом, tasks_api/auth_api/products_api, auth_token, authenticated_client, test_task (с очисткой), test_user, faker
tests/test_auth.py	10 тестов аутентификации: логин, негативный параметризованный логин, регистрация, дубликат email, /users/me с токеном и без, полный цикл
tests/test_tasks.py	11 тестов задач: CRUD, негативная валидация создания (параметризованная), несуществующая задача, фильтрация по статусу, полный жизненный цикл
tests/test_products.py	12 тестов товаров: список, фильтры по категории (параметризованные), наличию (граничный товар stock=0), цене (граничные диапазоны), по ID, несуществующий товар
utils/logger.py	настройка логирования (корневой логгер, вывод в stdout, формат с временем и уровнем)
utils/helpers.py	wait_for_server — ожидание готовности сервера по /api/v1/health (поллинг с таймаутом)
pytest.ini	конфигурация: testpaths, pythonpath, маркеры (smoke, negative, slow, integration), addopts с генерацией reports/report.html и reports/junit.xml
requirements.txt	зависимости проекта (requests, pytest, pydantic, pytest-html, pytest-xdist, python-dotenv, Faker, flask, flask-cors)
run_tests.cmd	лаунчер полного прогона и smoke-подмножества (py -m pytest tests/ -v, py -m pytest tests/ -m smoke -v)

Итого — 33 автоматизированных теста (10 — аутентификация, 11 — задачи, 12 — товары). Негативные сценарии помечены маркером negative, дымовые — smoke.

Клиент для работы с API (api/client.py)

Ядро клиента — класс APIClient: обёртка над requests.Session с базовым URL, таймаутом по умолчанию, установкой токена Bearer в заголовки сессии и логированием каждого запроса/ответа через hook сессии. HTTP-ошибки (400/401/404/409) сознательно не

превращаются в исключения: статус и тело ошибки проверяют сами тесты — иначе негативные сценарии падали бы не дойдя до проверок. Поверх клиента построены доменные обёртки TasksAPI, AuthAPI и ProductsAPI, инкапсулирующие состав запросов и параметров каждого домена (в т. ч. create_task с пустым телом для негативной проверки).

```
# -*- coding: utf-8 -*-
```

```
"""
```

```
Клиент для работы с тестовым API (JP4).
```

```
Обертка над requests: управление сессией, заголовками и аутентификацией,
```

```
логирование запросов и ответов. Методы возвращают объект requests.Response,
```

```
статусы (в том числе 400/401/404/409) проверяются в самих тестах.
```

```
"""
```

```
import logging
```

```
from typing import Optional, Dict
```

```
import requests
```

```
from requests.exceptions import RequestException
```

```
logger = logging.getLogger(__name__)
```

```
class APIClient:
```

```
    """Клиент для работы с REST API"""
```

```
    def __init__(self, base_url: str, timeout: int = 10):
```

```
        self.base_url = base_url.rstrip('/')
```

```
        self.timeout = timeout
```

```
        self.session = requests.Session()
```

```
        self.token: Optional[str] = None
```

```
        # Настройка логирования через hook на все ответы сессии
```

```
self.session.hooks['response'].append(self._log_response)
```

```

def _log_response(self, response: requests.Response, *args,
**kwargs):
    """Логирование HTTP-запросов и ответов"""
    logger.info(f"Request: {response.request.method}
{response.request.url}")
    logger.info(f"Request body: {response.request.body}")
    logger.info(f"Response status: {response.status_code}")
    logger.info(f"Response body: {response.text[:500]}...")
    return response

def set_auth_token(self, token: str):
    """Установка токена аутентификации"""
    self.token = token
    self.session.headers.update({'Authorization': f'Bearer
{token}'})

def _request(self, method: str, endpoint: str, **kwargs) ->
requests.Response:
    """Выполнение HTTP-запроса"""
    url = f"{self.base_url}{endpoint}"

    # Установка таймаута по умолчанию
    if 'timeout' not in kwargs:
        kwargs['timeout'] = self.timeout

    try:
        response = self.session.request(method, url,
**kwargs)
    except RequestException as e:
        logger.error(f"Request failed: {e}")
        raise

    # HTTP-ошибки не превращаем в исключения: статус
    проверяет тест
    if response.status_code >= 400:
        logger.warning(

```

```

        f"HTTP {response.status_code} for {method}"
{endpoint}: "
        f"{response.text[:300]}"
    )
    return response

    def get(self, endpoint: str, params: Optional[Dict] = None,
**kwargs) -> requests.Response:
        return self._request('GET', endpoint, params=params,
**kwargs)

    def post(self, endpoint: str, json: Optional[Dict] = None,
**kwargs) -> requests.Response:
        return self._request('POST', endpoint, json=json,
**kwargs)

    def put(self, endpoint: str, json: Optional[Dict] = None,
**kwargs) -> requests.Response:
        return self._request('PUT', endpoint, json=json,
**kwargs)

    def patch(self, endpoint: str, json: Optional[Dict] = None,
**kwargs) -> requests.Response:
        return self._request('PATCH', endpoint, json=json,
**kwargs)

    def delete(self, endpoint: str, **kwargs) ->
requests.Response:
        return self._request('DELETE', endpoint, **kwargs)

class TasksAPI:
    """Клиент для работы с API задач"""

    def __init__(self, client: APIClient):
        self.client = client

    def get_all_tasks(self, completed: Optional[bool] = None,
user_id: Optional[int] = None):

```

```

        params = {}
        if completed is not None:
            params['completed'] = str(completed).lower()
        if user_id is not None:
            params['userId'] = user_id
        return self.client.get('/api/v1/tasks', params=params)

    def create_task(self, title: Optional[str] = None,
completed: bool = False, user_id: int = 1):
        # title=None означает "пустое тело запроса" (проверка
негативного сценария)
        if title is None:
            return self.client.post('/api/v1/tasks', json={})
        data = {
            'title': title,
            'completed': completed,
            'userId': user_id
        }
        return self.client.post('/api/v1/tasks', json=data)

    def get_task(self, task_id: int):
        return self.client.get(f'/api/v1/tasks/{task_id}')

    def update_task(self, task_id: int, title: str, completed:
bool):
        data = {
            'title': title,
            'completed': completed
        }
        return self.client.put(f'/api/v1/tasks/{task_id}',
json=data)

    def delete_task(self, task_id: int):
        return self.client.delete(f'/api/v1/tasks/{task_id}')

class AuthAPI:

```

```

"""Клиент для работы с API аутентификации"""

def __init__(self, client: APIClient):
    self.client = client

def login(self, email: str, password: str):
    data = {
        'email': email,
        'password': password
    }
    return self.client.post('/api/v1/auth/login', json=data)

def register(self, email: str, password: str, name: str =
"""):
    data = {
        'email': email,
        'password': password,
        'name': name
    }
    return self.client.post('/api/v1/auth/register',
json=data)

def logout(self):
    return self.client.post('/api/v1/auth/logout')

def get_current_user(self):
    return self.client.get('/api/v1/users/me')

class ProductsAPI:
    """Клиент для работы с API товаров"""

    def __init__(self, client: APIClient):
        self.client = client

    def get_products(self, category: Optional[str] = None,

```

```

        min_price: Optional[float] = None,
        max_price: Optional[float] = None,
        in_stock: Optional[bool] = None):
    params = {}
    if category is not None:
        params['category'] = category
    if min_price is not None:
        params['minPrice'] = min_price
    if max_price is not None:
        params['maxPrice'] = max_price
    if in_stock is not None:
        params['inStock'] = str(in_stock).lower()
    return self.client.get('/api/v1/products',
params=params)

def get_product(self, product_id: int):
    return self.client.get(f'/api/v1/products/{product_id}')

```

Фикстуры PyTest (tests/conftest.py)

Сессионная фикстура `api_client` создаёт клиент после ожидания готовности сервера (`wait_for_server` по `/api/v1/health`) и закрывает сессию по завершении прогона. `Autouse`-фикстура `reset_data` выполняет `POST /api/v1/reset` перед каждым тестом — это обеспечивает изоляцию и идемпотентность: каждый тест стартует с предсказуемым состоянием данных. Фикстура `test_task` создаёт задачу через API и удаляет её после теста (очистка данных); `authenticated_client` устанавливает Bearer-токен и снимает его после теста; `faker` (`Faker`, `ru_RU`) генерирует правдоподобные тестовые данные.

```
# -*- coding: utf-8 -*-
"""
Фикстуры pytest для тестов API
"""

import pytest
from faker import Faker

from api.client import APIClient, TasksAPI, AuthAPI, ProductsAPI
from api.schemas import Task, AuthResponse
from utils.logger import setup_logging
from utils.helpers import wait_for_server

BASE_URL = "http://localhost:5000"

@pytest.fixture(scope="session", autouse=True)
def _logging():
    """Инициализация логирования на всю тестовую сессию"""
    setup_logging()

@pytest.fixture(scope="session")
def faker():
    """Генератор тестовых данных"""
    return Faker('ru_RU')
```

```

@pytest.fixture(scope="session")
def api_client():
    """Клиент для работы с API (с ожиданием готовности сервера)"""
    wait_for_server(BASE_URL)
    client = APIClient(base_url=BASE_URL)
    yield client
    client.session.close()

@pytest.fixture(autouse=True)
def reset_data(api_client):
    """Сброс данных перед каждым тестом (POST /api/v1/reset).

    Обеспечивает изолированность и идемпотентность тестов:
    каждый тест работает с предсказуемым начальным состоянием.
    """
    response = api_client.post('/api/v1/reset')
    assert response.status_code == 200
    yield

@pytest.fixture
def tasks_api(api_client):
    """API для работы с задачами"""
    return TasksAPI(api_client)

@pytest.fixture
def auth_api(api_client):
    """API для аутентификации"""
    return AuthAPI(api_client)

```

```

@pytest.fixture
def products_api(api_client):
    """API для работы с товарами"""
    return ProductsAPI(api_client)


@pytest.fixture
def auth_token(auth_api):
    """Получение токена аутентификации"""
    response = auth_api.login("student@university.ru",
"password123")
    auth_data = AuthResponse.model_validate(response.json())
    return auth_data.token


@pytest.fixture
def authenticated_client(api_client, auth_token):
    """Клиент с установленным токеном аутентификации"""
    api_client.set_auth_token(auth_token)
    yield api_client
    # Сбрасываем токен после теста
    api_client.token = None
    api_client.session.headers.pop('Authorization', None)


@pytest.fixture
def test_task(tasks_api, faker):
    """Создание тестовой задачи и её удаление после теста"""
    title = faker.sentence(nb_words=4)
    response = tasks_api.create_task(title=title)
    task_data = Task.model_validate(response.json())

    yield task_data

    # Удаляем задачу после теста (очистка)
    try:

```

```
        tasks_api.delete_task(task_data.id)
    except Exception:
        pass # Игнорируем ошибки при удалении
```

```
@pytest.fixture
def test_user(auth_api, faker):
    """Создание тестового пользователя"""
    email = faker.email()
    password = faker.password()
    name = faker.name()

    response = auth_api.register(email, password, name)
    user_id = response.json()['userId']

    return {
        'email': email,
        'password': password,
        'name': name,
        'id': user_id
    }
```

Тесты (tests/)

Тесты сгруппированы по доменам API. `test_auth.py`: успешный логин, параметризованный негативный логин (несуществующий email, неверный пароль, пустой email, пустой пароль), регистрация нового пользователя, дубликат email (409), получение `/users/me` с токеном и без (401), полный цикл «регистрация → вход → получение данных». `test_tasks.py` (листинг ниже): список задач, создание, параметризованная негативная валидация заголовка, получение по ID, несуществующая задача (404), обновление, удаление с проверкой, фильтрация по статусу, полный жизненный цикл Create → Read → Update → Delete. `test_products.py`: список, параметризованная фильтрация по категориям (включая несуществующую), наличие (граничный случай — товар «Футболка» с `stock=0` исключается), фильтрация по цене тремя граничными диапазонами, получение по ID, несуществующий товар (404). Ответы валидируются `pydantic`-схемами: тест проверяет не только статус, но и структуру тела и значения полей.

```
# -*- coding: utf-8 -*-
"""
Автоматизированные тесты для API задач
"""

import pytest

from api.schemas import Task, TaskList, DeleteTaskResponse,
ErrorResponse

class TestTasksAPI:
    """Тесты для эндпоинтов задач"""

    @pytest.mark.smoke
    def test_get_all_tasks(self, tasks_api):
        """Тест получения списка задач"""
        response = tasks_api.get_all_tasks()
        assert response.status_code == 200

        task_list = TaskList.model_validate(response.json())
        assert task_list.count >= 0
```

```

        assert isinstance(task_list.tasks, list)

def test_create_task_success(self, tasks_api, faker):
    """Тест успешного создания задачи"""
    title = faker.sentence()

    response = tasks_api.create_task(title=title)
    assert response.status_code == 201

    task = Task.model_validate(response.json())
    assert task.id > 0
    assert task.title == title
    assert task.completed is False

    # Очистка
    tasks_api.delete_task(task.id)

@pytest.mark.negative
@pytest.mark.parametrize("invalid_title, expected_error", [
    ("", "Поле 'title' обязательно и не может быть пустым"),
    (None, "Требуется JSON тело запроса"),
])
def test_create_task_validation(self, tasks_api,
invalid_title, expected_error):
    """Тест валидации при создании задачи (параметризованный
негативный)"""
    data = {}
    if invalid_title is not None:
        data['title'] = invalid_title

    response = tasks_api.create_task(**data)
    assert response.status_code == 400

    error = ErrorResponse.model_validate(response.json())
    assert expected_error in error.error

```

```

def test_get_task_by_id(self, tasks_api, test_task):
    """Тест получения задачи по ID"""
    response = tasks_api.get_task(test_task.id)
    assert response.status_code == 200

    task = Task.model_validate(response.json())
    assert task.id == test_task.id
    assert task.title == test_task.title

@pytest.mark.negative
def test_get_nonexistent_task(self, tasks_api):
    """Тест получения несуществующей задачи"""
    response = tasks_api.get_task(99999)
    assert response.status_code == 404

    error = ErrorResponse.model_validate(response.json())
    assert "не найдена" in error.error.lower()

def test_update_task(self, tasks_api, test_task, faker):
    """Тест обновления задачи"""
    new_title = faker.sentence()

    response = tasks_api.update_task(
        task_id=test_task.id,
        title=new_title,
        completed=True
    )
    assert response.status_code == 200

    updated_task = Task.model_validate(response.json())
    assert updated_task.title == new_title
    assert updated_task.completed is True

def test_delete_task(self, tasks_api, faker):
    """Тест удаления задачи"""
    # Создаем задачу для удаления

```

```

title = faker.sentence()
response = tasks_api.create_task(title=title)
task_id = response.json()['id']

# Удаляем задачу
response = tasks_api.delete_task(task_id)
assert response.status_code == 200

deleted =
DeleteTaskResponse.model_validate(response.json())
assert "удалена" in deleted.message.lower()

# Проверяем, что задача действительно удалена
response = tasks_api.get_task(task_id)
assert response.status_code == 404

@pytest.mark.parametrize("completed_filter", [True, False])
def test_filter_tasks_by_status(self, tasks_api,
completed_filter):
    """Тест фильтрации задач по статусу
    (параметризованный) """
    response =
tasks_api.get_all_tasks(completed=completed_filter)
    assert response.status_code == 200

    task_list = TaskList.model_validate(response.json())

    # Проверяем, что все задачи соответствуют фильтру
    for task in task_list.tasks:
        assert task.completed == completed_filter

def test_task_lifecycle(self, tasks_api, faker):
    """Полный жизненный цикл задачи: Create -> Read ->
Update -> Delete"""
    # 1. Create
    title1 = faker.sentence()
    response = tasks_api.create_task(title=title1)

```

```
assert response.status_code == 201
task_id = response.json()['id']

# 2. Read (проверяем создание)
response = tasks_api.get_task(task_id)
assert response.status_code == 200
assert response.json()['title'] == title1

# 3. Update
title2 = faker.sentence()
response = tasks_api.update_task(task_id, title2, True)
assert response.status_code == 200
assert response.json()['title'] == title2
assert response.json()['completed'] is True

# 4. Delete
response = tasks_api.delete_task(task_id)
assert response.status_code == 200

# 5. Verify deletion
response = tasks_api.get_task(task_id)
assert response.status_code == 404
```

Запуск тестов и результаты

Порядок запуска: 1) старт сервера из ЛР2 (py lab2_api_server.py, порт 5000); 2) прогон из каталога ЛР4\api_tests командой py -m pytest (конфигурация pytest.ini: testpaths = tests, pythonpath = ., маркеры; addopts автоматически генерирует HTML- и JUnit-отчёты). Полный текст вывода прогона сохранён в ЛР4\pytest_output.txt.

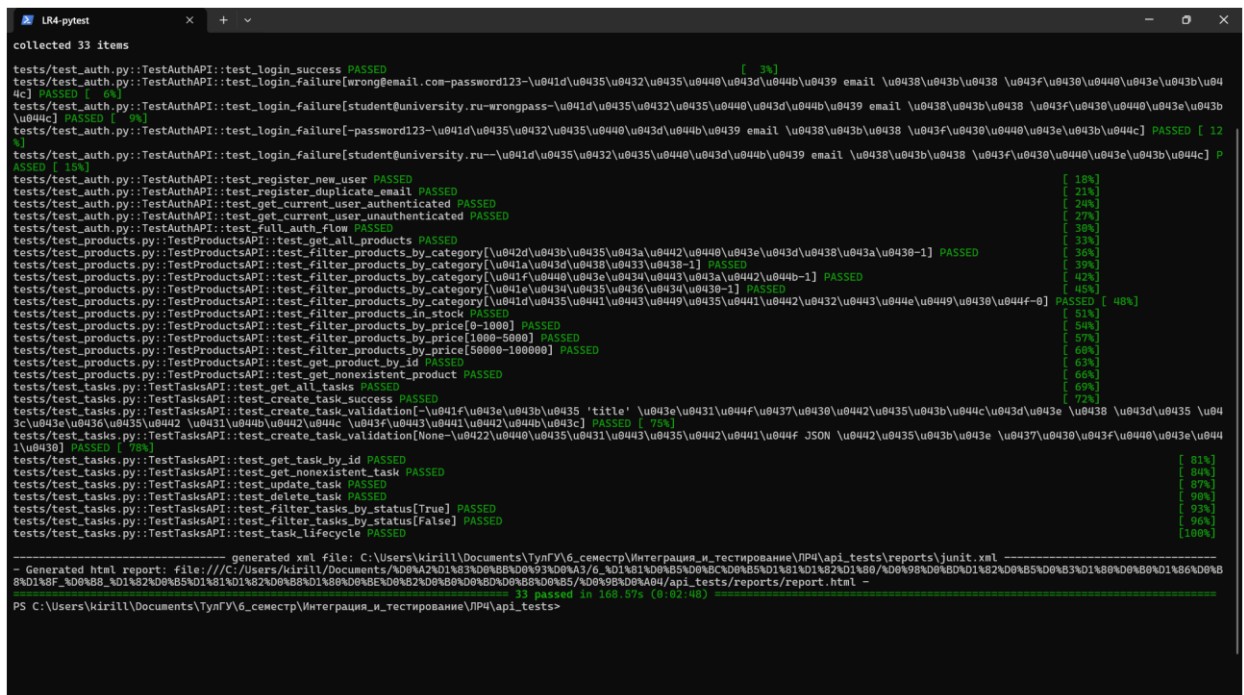


Рисунок 1 — Финальный прогон: collected 33 items, 33 passed (168.57 c)

Результат: собрано 33 теста — 33 passed, 0 failed, 0 skipped. Время прогона — около 169 секунд (задержки обусловлены валидацией и ожиданиями ответов сервера). Отдельно проверено smoke-подмножество (py -m pytest tests/ -m smoke).

HTML-отчёт pytest-html

Плагин `pytest-html` (addopts: `--html=reports/report.html --self-contained-html --junitxml=reports/junit.xml`) формирует автономный HTML-отчёт: сводка (33 passed / 0 failed), параметры окружения, таблица результатов с фильтрацией по исходу и раскрываемыми деталями каждого теста (включая перехваченные логи запросов/ответов). JUnit XML (`reports/junit.xml`) предназначен для интеграции с CI.

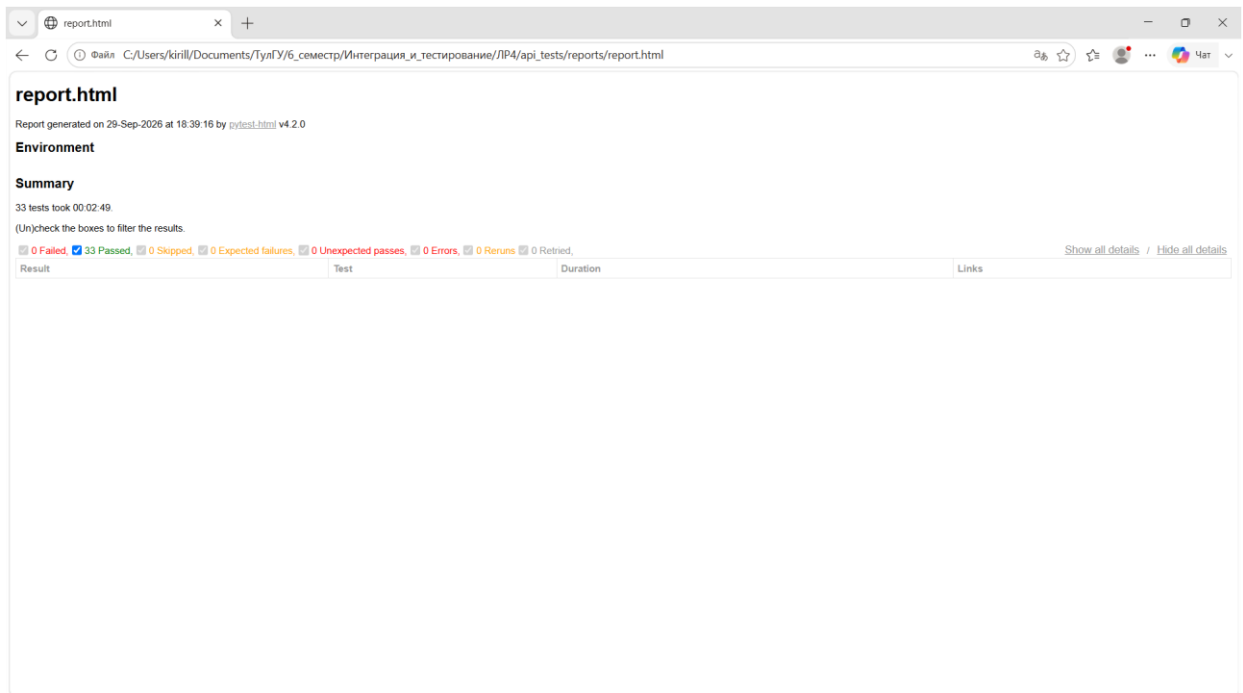


Рисунок 2 — HTML-отчёт reports/report.html в браузере (Summary: 33 Passed, 0 Failed)

Выводы

Автоматизировано тестирование REST API сервера из ЛР2: спроектирован и реализован проект из 33 тестов (10 — аутентификация, 11 — задачи, 12 — товары), покрывающий позитивные, негативные, граничные и параметризованные сценарии, а также смену состояний (полный CRUD-цикл задачи). Построена типовая архитектура API-тестирования: клиент-обёртка requests с логированием и управлением токеном, доменные API-классы, pydantic-схемы для проверки контрактов ответов, фикстуры с autouse-сбросом данных (изоляция тестов) и очисткой созданных сущностей, генерация данных Faker. Ключевое проектное решение — не превращать HTTP-ошибки в исключения на уровне клиента: статусы 400/401/404/409 проверяются в тестах, что позволило негативным сценариям проверять и код, и тело ошибки (сообщения сервера валидируются схемой ErrorResponse и точными подстроками). В ходе работы ожидания двух тестов были приведены к фактическому поведению сервера: в /auth/login отсутствует проверка формата email (она есть только в /auth/register), поэтому пустой email даёт 401 «Неверный email или пароль»; ответ /users/me без токена — «Требуется аутентификация». Отчётность: полный вывод прогона сохранён в pytest_output.txt, HTML-отчёт (reports/report.html) и JUnit XML (reports/junit.xml) генерируются автоматически при каждом запуске

— итог: 33 passed, 0 failed. Полученная структура переиспользуема: подключение нового домена API сводится к добавлению класса-обёртки, схем и модуля тестов.